

Sun Jingfeng

Phone 195-2460-3214 (WeChat) | Email jlu_sunjingfeng@163.com | Age 31 | Experience 4 Years | Location Beijing
Target Role AI Agent Java Engineer | Expected Salary 27k | English Working Proficiency

Education

Jilin University (Project 985) · B.Eng. in Software Engineering 2014-2020

Work Experience

Beijing Hongxiang Technology Co., Ltd. Shared Development Center · Team Lead, R&D Group 1 2023.02 – 2025.12
Beijing Bailian Intelligence Technology Co., Ltd. Engineering Department · Front-End Engineer 2021.12 – 2022.12

Professional Summary

- Led end-to-end delivery of mid- to large-scale projects, spanning solution design, development, deployment, and production rollout across front-end, back-end, and AI agent capabilities.
- Improved reuse through component abstraction, layered architecture, and shared modules, while continuously optimizing front-end performance, API latency, and service stability.
- Drove technology selection, project bootstrapping, coding standards, microservice architecture, and deployment setup, while introducing AI-assisted development practices into the engineering workflow.

Technical Skills

AI Agents & AI Coding

- Built AI agent services independently and integrated them with Java microservices in private-network environments.
- Hands-on experience with LangChain and private-model integration, with working knowledge of LangGraph for complex workflow orchestration.
- Proficient with AI coding tools such as Cursor, Copilot, and Claude Code to accelerate implementation, debugging, and troubleshooting.
- Skilled in prompt engineering and AI rule configuration, including Rules, Skills, and MCP, with experience defining reusable prompts and project-level conventions.

Front End

- Strong command of TypeScript, JavaScript, HTML5, and CSS3, including ES6+ features and asynchronous programming.
- Hands-on experience with Vue 3, Vue 2, Vue Router, and Pinia/Vuex, with solid understanding of reactivity internals and compilation optimization.
- Hands-on experience with React, React Router, and Redux/RTK, with solid understanding of Fiber, virtual DOM, and diffing principles.
- Comfortable with mainstream libraries including Element Plus, Ant Design, ECharts, and Leaflet, with the ability to ramp up quickly on unfamiliar frameworks.
- Solid front-end engineering practices covering package management, build tooling, and code quality, including pnpm/npm, Vite/Webpack, ESLint, Stylelint, and Prettier.
- Strong in component-driven development and logic reuse through custom composables and hooks, improving modularity and maintainability.
- Experienced with Chrome Performance, Memory, and Lighthouse for diagnosing bottlenecks, analyzing memory usage, and applying targeted optimizations.
- Production experience with micro-frontends and iframe-based multi-application integration, including SSO, permission control, style isolation, and cross-application communication.

Back End

- Proficient in Java, including collections, Stream API, I/O, multithreading, and network programming.
- Hands-on experience with Spring Boot and Maven, with solid understanding of auto-configuration, AOP, and Spring Security-based JWT authentication with endpoint-level access control.
- Strong working knowledge of MySQL and MyBatis-Plus, including indexing, transaction management, and dynamic SQL.
- Practical experience with Redis and its common data structures for caching, session management, and related scenarios.

- Familiar with microservice architecture and Spring Cloud components including Nacos, OpenFeign, Gateway, Sentinel, and Seata.
- Experience using RabbitMQ for asynchronous decoupling, traffic smoothing, and common messaging patterns.
- Hands-on experience with Docker, Nginx, Git, and Jenkins, including containerized deployment using Dockerfile and Docker Compose.
- Comfortable with Linux operations, server setup, routine maintenance, and API performance validation with JMeter.
- Working knowledge of Python, FastAPI, and Pydantic for service development and delivery.

Project Experience

Meteorological Radar Monitoring & Early Warning System Series · Originally delivered for Tianjin 2025.02 – 2025.12

and later extended into multiple provincial editions and supporting management systems for Liaoning, Jiangsu, Shanxi, and Ningxia, with an added AI agent capability for weather warning message generation.

AI Agent Stack: [Python](#), [FastAPI](#), [Pydantic](#), [LangChain](#)

- Built an AI agent for weather warning message generation as an independent Python microservice and integrated it into the existing Java service ecosystem.
- Implemented weather warning message generation based on time, meteorological factors, and map-viewport latitude/longitude ranges, and completed end-to-end front-end and back-end integration.
- Assembled business context, queried raw domain data, filtered it by viewport range, and passed structured inputs to the model.
- Used tool calling for deterministic tasks such as reverse geocoding to prevent hallucinated place names and improve output accuracy and explainability.
- Standardized model invocation, tool orchestration, result parsing, and API schemas to improve the maintainability and reliability of agent capabilities across the project.

Front-End Stack: [React](#), [Immer](#), [React Router](#), [Redux](#), [Ant Design](#), [TypeScript](#), [Vite](#), [React Compiler](#)

- **Established a parent-child application architecture**
 - Eliminated stale asset issues caused by differing browser cache behavior after deployment updates.
 - Implemented screenshot capture across iframe boundaries.
 - Unified light/dark theme switching across parent and child applications.
 - Designed a permission model that resolved permission-key conflicts across multiple applications and enabled child apps to synchronize permissions independently.
 - Implemented unified authentication and SSO across parent and child applications.
- Built map interaction utilities for screenshot capture, distance measurement, area drawing, and point-of-interest workflows.
- Authored TypeScript declaration files for an internal map package and published it as a private npm package.
- Fixed an edge-case bug in GIF generation under extremely slow network conditions.
- Built an ultra-long timeline component with time selection and playback controls.
- Implemented synchronized dual-map interaction.
- Designed a multi-province adaptation strategy that maximized reuse of shared logic while isolating province-specific behavior.
- Implemented configurable legend color editing.

Back-End Stack: [Spring Boot](#), [Maven](#), [Spring Cloud \(Nacos, OpenFeign, Gateway, Sentinel\)](#), [Spring Security](#), [MyBatis-Plus](#), [MySQL](#), [Redis](#), [RabbitMQ](#), [Docker](#), [JMeter](#)

- Developed RESTful APIs with Spring Boot for radar data queries, weather warnings, file management, and other business modules.
- Built a Spring Cloud microservice architecture that reused core capabilities across provinces while supporting province-specific extensions.
- Used Nacos for service discovery and centralized configuration, and OpenFeign for declarative service-to-service communication.
- Used Gateway for unified routing and authorization, and implemented centralized authentication and endpoint-level access control based on Spring Security + JWT + RBAC, supporting parent-child application SSO.
- Processed meteorological data with thread pools and wrote the results to Redis cache to improve throughput and reduce database pressure.
- Identified bottlenecks through JMeter load testing and combined Sentinel rate limiting with fallback strategies to preserve stability under high concurrency.
- Used RabbitMQ for asynchronous radar data pre-processing and warning notification delivery to improve responsiveness.
- Orchestrated all microservices and middleware with Docker Compose and completed containerized deployment on Linux servers.

Leisheng X Single-Station Display System Optimization · Took over a live single-station product and focused on performance and codebase optimization. 2025.01

Tech Stack: [Vue 3](#), [Vue Router](#), [Pinia](#), [Element Plus](#), [Leaflet](#), [TypeScript](#), [Vite](#)

Performance Optimization

- Used shallow reactive APIs to reduce unnecessary deep reactivity overhead.
- Pre-cached raster overlay assets to improve initial render speed.
- Added pause and resume support for cached tasks to avoid UI freezes caused by main-thread blocking.
- Enabled component caching for high-complexity pages with frequent switching to improve navigation smoothness.
- Implemented multi-resolution responsiveness with postcss-px-to-viewport.

Codebase Optimization

- Standardized code conventions with ESLint, Stylelint, Prettier, and EditorConfig.
- Managed environment-specific differences through environment variables.
- Refined component boundaries and extracted shared constants and styles.

Huai'an Smart Meteorological Integrated Service Platform · Led front-end delivery for map visualization, permission management, and theming capabilities. 2024.08 – 2024.12

Tech Stack: [Vue 3](#), [Vue Router](#), [Pinia](#), [Element Plus](#), [Leaflet](#), [Highcharts](#), [TypeScript](#), [Vite](#)

- Established the foundational permission framework with route-level and page-level access control, including one-click synchronization of the latest permission configuration.
- Built reusable map composables for multiple pages, supporting light/dark theme switching and configurable manual loading.
- Implemented light/dark theme switching based on CSS variables.
- Built in-page screenshot features supporting area selection, image download, and clipboard copy.
- Reused the Element Plus Message mounting approach by appending DOM nodes to the body to guarantee top-layer overlay behavior.
- Fixed incorrect child-page rendering under multi-level route caching scenarios.
- Used render functions and advanced Vue APIs to render components as map markers.
- Identified bottlenecks through the Performance panel and optimized long-running tasks with generator functions and Web Workers.

Hefei Smart Agriculture Meteorological Data Platform · Developed front-end modules for maps, charts, and data presentation features. 2024.03 – 2024.07

Tech Stack: [Vue 3](#), [Vue Router](#), [Pinia](#), [Element Plus](#), [Leaflet](#), [Highcharts](#), [TypeScript](#), [Vite](#)

- Built a clear cross-component data flow model using defineExpose, provide, and props to support module-level coordination.
- Decoupled map boundaries, station layers, and raster overlays through composables.
- Used dynamic components and shallow reactive data to make page structure adapt to changing data.
- Implemented a region selector linked to the map with automatic focus and target highlighting.
- Batched ordered watchers to merge raster overlay updates triggered by multiple dependent parameters and avoid redundant rendering.
- Built progressive chart components inspired by class-inheritance patterns to reuse bar, pie, and line chart implementations.

Vue 3 Component Library · An internal component library created to accumulate reusable components and complex UI patterns and improve team productivity. 2024.02

Tech Stack: [VitePress](#)

- Evaluated VuePress, VitePress, Storybook, and Docusaurus, and selected VitePress after comparison.
- Set up the initial framework and integrated Vue, Element Plus, Sass, and other core dependencies.
- Configured the homepage, custom theme, documentation structure, and search.
- Used vitepress-theme-demoblock to present component demos alongside source code.
- Resolved style leakage issues with postcss-prefix-selector and :not selectors.
- Used dynamic imports to resolve browser-only API availability issues in SSR environments.
- Built a Node.js-based file scanning process to auto-generate grouped and ordered navigation configuration.
- Implemented authentication and theme-color switching through a custom theme.
- Used Terser to minify and obfuscate code and remove console statements in production.

- Wrote usage documentation covering Markdown syntax, component publishing workflows, and conventions.

Intelligent Meteorological Integrated Platform · Owned alerting and file management modules, along with shared component abstraction and data-driven development for multiple alert page types. 2023.11 – 2024.01

Tech Stack: [Vue 3](#), [Vue Router](#), [Pinia](#), [Element Plus](#), [TypeScript](#), [Vite](#)

- Built reusable components for page layouts, paginated lists, and file operations to improve delivery efficiency across similar pages.
- Built data-driven views so one codebase could support create and edit pages for multiple alert file types.
- Split business logic to balance reuse of shared logic with flexibility for variant-specific behavior.

UHV Dense Corridor Meteorological Service Platform · Owned map visualization, risk corridor rendering, marker presentation, and performance optimization. 2023.02 – 2023.10

Tech Stack: [Vue 3](#), [Vue Router](#), [Pinia](#), [Element Plus](#), [Leaflet](#), [Highcharts](#), [TypeScript](#), [Vite](#)

- Rendered color-coded GeoJSON corridors and feature markers on the map based on risk-level data, with hover and click popups.
- Applied lazy-loading and object-pool strategies to reuse frequently changing map markers and reduce memory usage.
- Used render and h functions to parse single-file components and dynamically render marker popup content.
- Built page-level components and passed parameters through route metadata to reuse multiple similar pages.
- Built map-related composables to unify rendering logic across multiple pages.

Vue 3 Project Starter Framework · During the department-wide technology migration from Vue 2 to Vue 3, Vuex to Pinia, JavaScript to TypeScript, and Webpack to Vite, I built the next-generation project foundation. 2023.02

Tech Stack: [Vue 3](#), [Vue Router](#), [Pinia](#), [TypeScript](#), [Vite](#)

- Bootstrapped the initial project with create-vue and integrated core dependencies such as Vue Router, Pinia, and Element Plus.
- Referenced best practices from open-source admin frameworks including vue-element-admin, vue-pure-admin, and vue-vben-admin.
- Integrated unplugin-auto-import and unplugin-vue-components for automatic API and component imports.
- Configured ESLint, Stylelint, Prettier, and EditorConfig to enforce consistent team-wide coding standards.
- Completed project configuration for environment variables, request proxying, path aliases, CSS preprocessors, TypeScript, and npm.
- Wrote foundational code including route guards, request interceptors, TypeScript utilities, and global styles.
- Provided VS Code configuration with recommended extensions and editor settings to reduce onboarding time for the team.

Profile

- Full-stack engineer with hands-on experience across Vue, React, and Java, combining AI agent delivery, system integration, and team leadership.
- Familiar with engineering approaches that combine business systems with intelligent-agent capabilities, and able to apply AI agents and AI-assisted development in real production scenarios.